

Machine Learning Exercise 3B

Problem 1 *In the previous exercise, you used logistic regression to classify Fashion MNIST images. However, logistic regression is a linear classifier and cannot capture complex patterns. In this exercise, you will implement neural networks using TensorFlow/Keras and explore how network architecture affects performance.*

You will use the same Fashion MNIST dataset as before, which contains 28×28 grayscale images of clothing items in 10 categories.

i) *Load the Fashion MNIST dataset and preprocess it:*

- *Normalize pixel values to $[0, 1]$*
- *For dense networks: flatten images to 784-dimensional vectors*
- *For CNN: reshape images to $28 \times 28 \times 1$ (adding channel dimension)*
- *Convert labels to one-hot encoding using `to_categorical`*
- *Split the training set into training (50000 samples) and validation (10000 samples)*

ii) *Create a **simple neural network** using the `Sequential` API with:*

- *Input layer: 784 units*
- *Hidden layer: 32 units with ReLU activation*
- *Output layer: 10 units with softmax activation*

iii) *Compile the model using:*

- *Optimizer: `adam`*
- *Loss function: `categorical_crossentropy`*
- *Metrics: `accuracy`*

iv) *Train the simple model for 50 epochs and record the training and validation accuracy.*

v) *Create a **deeper neural network** with:*

- *Input layer: 784 units*
- *Hidden layer 1: 256 units with ReLU activation*
- *Hidden layer 2: 128 units with ReLU activation*
- *Hidden layer 3: 64 units with ReLU activation*
- *Output layer: 10 units with softmax activation*

vi) *Train the deeper model for 50 epochs and record the training and validation accuracy.*

vii) *Create a **Convolutional Neural Network (CNN)** with:*

- *Input: $28 \times 28 \times 1$*
- *Conv2D: 32 filters, 3×3 kernel, ReLU activation*
- *MaxPooling2D: 2×2*
- *Conv2D: 64 filters, 3×3 kernel, ReLU activation*

- *MaxPooling2D: 2×2*
- *Flatten*
- *Dense: 64 units with ReLU activation*
- *Output: 10 units with softmax activation*

viii) Train the CNN for 50 epochs and record the training and validation accuracy.

ix) Plot the learning curves (training and validation accuracy vs. epochs) for all three models.

x) Create a summary table comparing all architectures:

- *Number of parameters*
- *Training accuracy*
- *Validation accuracy*
- *Test accuracy*
- *Training time*

xi) Evaluate all models on the test set. Answer the following questions:

- *Which architecture performs best?*
- *Why do CNNs perform better than fully-connected networks on image data?*
- *Compare the CNN with the Deep fully-connected network: which has more parameters? Which performs better?*
- *What is the accuracy improvement per parameter for each model?*